Yourbit AI v3

Project Documentation

Architecture and modules · plugin system · what can and cannot be changed · forks and mods · build and release

Application package (applicationId)	com.yourbitai.app
Code (namespace)	com.youraicqa.app
Version	3.0.0 (versionCode 5)
Platform	Android, minSdk 26, targetSdk 35
Language / UI	Kotlin, View-based UI (no Compose), Material Components
Native part	C++17, llama.cpp via JNI (ABIs: arm64-v8a, armeabi-v7a, x86_64)
Build	Gradle 8.x, AGP 8.7.3, Kotlin 2.0.21, JDK 17, NDK 27.0.12077973

This document describes the state of the code in the archive YourbitAIv3.zip. It is not legal advice.

1. What this project is

Yourbit AI (VonCure Studio) is an Android client application for working with language and image models. The app contains no built-in API keys or proxies: the user connects their own provider keys, their own OpenAI-compatible servers, or runs a GGUF model directly on the device. Requests go straight to the chosen provider, and chat history is stored on the device.

Main features

- Chat with streaming output, Markdown, code highlighting, LaTeX formulas, SVG and HTML preview, table and quiz cards.
- Built-in providers: OpenRouter, Hugging Face, Groq, Google AI Studio (Gemini), Cloudflare Workers AI, LLM7.io, Pollinations, Mistral, Cohere, OpenAI, Qwen, DeepSeek, Anthropic, NVIDIA NIM; plus any OpenAI-compatible server (Ollama, LM Studio, Automatic1111, etc.).
- Model modes: Chat, Vision, Image generation; video (6 frames, 3 in economy mode).
- Arena (up to 50 models answer a single request), Authority (models rate each other), Resolve (automatic switch to the next model on error or length limit).
- Agent: the AI controls a built-in browser (separate cookies, confirmation of dangerous actions).
- Sandboxed terminal, text and image editor, “Files” section, web search of up to 3 rounds.
- Token saving (chat memory, short answers, etc.), statistics, achievements, 8 interface languages.
- Chat and account export/import (AES-256 encryption); cloud sync of the device list only.
- Plugins (.plugin): themes, quick actions, custom AI providers, panels, network requests, and more.

Repository structure

Path	Purpose
app/src/main/java/com/youraicqa/app	All Kotlin application code (≈112 files, ≈36,000 lines)
app/src/main/java/.../local	Local GGUF engine: model loading, generation service, prompt formatting
app/src/main/java/.../linkimporter	Importing media from links, saving to “Downloads”
app/src/main/cpp	CMakeLists.txt and llama_bridge.cpp (JNI bridge to llama.cpp)
app/src/main/assets/lang	Translations: en, es, pt, kk, uk, be, zh (*.tsv)
app/src/main/assets/fonts	UI / achievements font
app/src/test	Unit tests (plugins, network, server, errors, response merging)
supabase/schema.sql	SQL schema for cloud session storage (RLS with no policies, access via yb_* functions)

Path	Purpose
docs/example-openai-compatible.plugin	Example plugin: OpenAI-compatible provider
.github/workflows/build-apk.yml (separate)	CI build of the APK

The folder app/src/main/cpp/llama.cpp is not included in the archive: it is cloned at build time (see section 7).

2. Modules and their responsibilities

Files are grouped by responsibility. Sizes are approximate, by line count.

Core and shared framework

File / class	Responsibility
NanoApp	Application: app start, initialization of shared services.
NanoBaseActivity	Base for all screens: language, font, theme, applying the plugin theme to the window, entrance animations.
MainActivity (≈7,400 lines)	Main chat screen: input, attachments, sending, streaming reception, web search, tools, model selection, regenerate/continue, calling plugin hooks (onMessage/onRender), showing plugin UI extensions.
ChatsActivity, SessionsManager	Chat list; SessionsManager — device identifier and label, account session list.
Splash/Welcome/RestartActivity	Splash screen, first launch, app restart after a language/theme change.
Loc	Localization: the Russian text is the translation key (Loc.t); Loc.pick(ru, en) for pairs.
Th, UiPrefs, Glass, ViewAnimExt	Colors and light/dark theme adaptation, user UI settings, glass effect, press animations.

Models, providers, network

File / class	Responsibility
ApiClient	Requests to cloud providers: model list, chat, image generation (incl. Cloudflare, FLUX), human-readable error messages.
ApiKeyGate, ApiKeysActivity	List of built-in providers and key presence check; key entry screen.
ProvidersStore, ModelsStore, ModelsActivity	Custom providers and models (Chat/Vision/Image modes), management screen.
BuiltinModels	List of built-in models: bundled + downloaded from a server, cached on the device.
ServerStore, ServerLogic, CustomServerClient, ServerError	Custom servers: storage (key encrypted with AES, bound to the server id), URL normalization, SSE and response parsing, errors, certificate check (CertChangedException).
Transport	“Address transport”: once an hour requests a JSON with controller and cloud addresses; accepts addresses only for allowed domains (val.run, supabase.co/.in). Stored locally in transport.json.
Cloud + supabase/schema.sql	Sync of the device/session list via RPC yb_sync, yb_revoke, yb_rotate, yb_delete_account, yb_check. Message text and keys are not sent to the cloud.

Modes and scenarios

File / class	Responsibility
ArenaActivity	Arena: several models answer one request; storage of turns and attachments.
ModelAuthorityActivity/Prefs	Authority: models rate answers; order and modes of built-in models (incl. image visibility).
ResolveActivity, ResolvePrefs	“Resolve”: on error/length limit the unfinished answer is passed to the next model in a cycle.

File / class	Responsibility
AgentActivity, AgentProtocol	Agent: “observation → action” loop in a WebView; parsing JSON actions, blocking local addresses and javascript:/file: schemes, confirmation of payment/deletion actions, ban on password autofill.
TerminalActivity	Terminal inside the app sandbox: ls/cd/cat/mkdir/rm/cp/mv, etc.; service commands /help, /addapikeys, /addmodels, /syncmodels, /logs, /sessions, /devmenu, /ai (a command from the AI runs only after confirmation).
ModesActivity	Modes menu (Arena, Agent, Coder — currently a “Coming soon” placeholder).
ChatTools, ContinueMerge	Parsing tool tags in the model response (call limit per response), saving files from blocks, cleaning the response; merging a continuation of a cut-off response without repeats.

Local model (GGUF)

File / class	Responsibility
local/LocalLlamaEngine	Loading .gguf (copy/download, signature check), parameters: context, threads, GPU layers, prompt format.
local/LlamaNative	JNI wrapper: nativeLoadModel and generation with an onToken callback.
local/LocalGenerationService/Holder	Foreground service (dataSync type) for background generation.
local/LocalPromptFormatter, LocalPromptTrim, ReplySanitizer	Prompt template, trimming history to fit the context, response cleaning.
cpp/llama_bridge.cpp	C++ bridge: tokenization, batches, sampler (temperature, top-p, repeat penalty), UTF-8-safe token delivery.

Content, files, editors

File / class	Responsibility
ChatMarkdown, MathFormat, MathSpans, CustomMathSymbols, MathSymbols*	Markdown and LaTeX rendering, custom symbols.
SyntaxHighlighter, SvgPreview, HtmlPreviewActivity, TableCardLayout, QuizCardView	Code highlighting; SVG without JS/network; HTML preview; table and quiz cards.
FileTextExtractor, OfficeDocumentCodec	Text from attachments and archives (ZIP/RAR/TAR/GZ in memory), DOCX and PDF (PDFBox-Android).
ChatAttach, ChatImages, FilesActivity, FilePreviewActivity	Attachments, chat images, the “Files” section and preview.
TextEditorActivity, ImageEditorActivity/CanvasView	Text editor; image editor.
linkimporter/*	Importing media from links, Fandom host list, saving to “Downloads” via MediaStore.

Data, economy, service

File / class	Responsibility
BackupActivity, BackupManager	Export/import of chats (.nanobackup encrypted, .md, .json) and account; merge modes; sessions.
EconomyActivity/Prefs	Chat memory, short answers, reasoning limit, light images, polling the model server less often, etc.
StatsActivity/Prefs, AchievementsActivity/Store, AchievementPopup	Request statistics (local) and achievements.
CustomizeActivity, ThemeEditorActivity	Theme customization; the theme editor produces JSON of the same form as a plugin theme.

File / class	Responsibility
DevMenuActivity, DevSettingsActivity, AdvancedSettingsActivity	Developer parameters: system prompt, persona, logs, advanced settings.
NanoLog, CrashReporter, AppErrorUi	Local log (keys and message texts are not written), crash reports, catalog of human-readable errors.
FaqActivity/Content, AboutActivity, ProfileStore	FAQ, “About”, user/assistant name and avatar.

3. Plugin system

A plugin is a single .plugin file: JSON with metadata and JavaScript code in the **script** field. The code runs in a hidden WebView and talks to the app through the **nano** object. All permissions are checked on the Kotlin side (PluginRuntime.Bridge) — a plugin cannot bypass them simply by calling the needed method.

3.1 File format

```
{
 "id": "my-plugin",
 "name": "My plugin",
 "description": "What it does",
 "author": "me",
 "version": "1.0",
 "permissions": ["UI_THEME", "QUICK_ACTIONS"],
 "capabilities": { "network": {"domains": ["api.example.com"]}, "files": true,
 "backgroundTask": true, "uiAction": true, "aiProvider": true },
 "script": "nano.setTheme({...});"
}
```

The **id** field is required: without it the file is rejected. Permissions can be declared as a **permissions** list and/or a **capabilities** block — the result is their union. Permission names are case-insensitive, and a dot is replaced with “_” (ui.theme → UI_THEME). If the resulting permission set is empty, the plugin does not start. A disabled plugin does not start either.

3.2 Installation and checks

- Only files with the .plugin extension are accepted; the manifest and script are limited to 200,000 bytes each.
- PluginScanner computes SHA-256 and looks for suspicious items: Java.perform/use/available, android.app.*, Class.forName, new Function(, eval(, document.write(, mentions of sharedpreferences, access to AndroidBridge; an escape-sequence share above 35%; base64 blocks longer than 400 characters.
- If anything is found, the user sees a warning and the “Install” button becomes “dangerous”, but installation is not blocked. That is, the scanner is advice to the user, not a prohibition.
- Before installation the user is offered to check the file on VirusTotal (the site opens; the external hash check in code is currently a stub).
- On installation the id, sha256 and permission list are written to the log; in the plugin list the permissions are shown to the human in Russian.

3.3 Sandbox

- Plugin WebView: JavaScript enabled, DOM Storage enabled, file and content access disabled, page about:blank.
- The only bridge into the app is nanoBridge; all methods first check the corresponding permission.
- Script errors do not crash the app: they are written to the log and shown in the chat (queue of up to 8 messages).
- A custom panel (CUSTOM_PANEL) is shown in a separate Activity where JavaScript is disabled — it is HTML/CSS only.
- All network requests go only through nano.net; direct fetch/XMLHttpRequest from the WebView is not separately allowed and is not a supported path.

3.4 Permissions

Permission	What it grants	API / effect
UI_THEME	Styling: colors, gradient, borders, rounding, entrance animation, glass, glow	nano.setTheme(obj)
QUICK_ACTIONS	Quick buttons in the chat	nano.addQuickAction(id, title)
MESSAGE_HOOKS	Processing the user's text before sending	nano.onMessage(fn(text) → string)
MESSAGE_RENDER	Processing the model's response before display	nano.onRender(fn(text) → string)
SEARCH_SOURCE	Custom web search source (source name)	nano.registerSearchSource(name)
CUSTOM_PANEL	Custom panel/screen in HTML without JS	nano.setPanelHtml(html)
APP_SETTINGS	Safe app settings from an allowlist	nano.setAppSetting(key, value)
SETTINGS	The plugin's own isolated storage (key → string)	nano.getSetting / setSetting
NETWORK	HTTP(S) requests to public addresses	nano.net.request(opts)
DOWNLOAD	File downloads (automatically also grants FILES)	nano.download.start(opts)
FILES	Saving files to the plugin folder	nano.files.save(opts)
BACKGROUND_TASK	Background tasks with progress and cancellation	nano.task.*
UI_ACTION	Custom actions and notifications	nano.ui.action.register, nano.ui.notify
UI_EXTEND	Embedding buttons/tabs/blocks into the interface	nano.ui.extend(opts)
AI_PROVIDER	Custom providers and models (receive chat history when their model is selected)	nano.ai.*

Implicit permissions: network.domains → NETWORK; download.domains → DOWNLOAD + FILES; files → FILES; backgroundTask → BACKGROUND_TASK; uiAction → UI_ACTION; aiProvider → AI_PROVIDER + NETWORK.

3.5 The nano API in detail

Theme (UI_THEME)

```
nano.setTheme({ colors: {
 accent: "#7C5CFF", background: "#0B0B0E",
 backgroundGradient: { to: "#1A1033", angle: 135 },
 cardBackground: "#16161C", cardBorderColor: "#2A2A35",
 cardBorderWidth: 1, bubbleCornerRadius: 18, bubbleEntrance: "scaleFade",
 headerBackground: "#0B0B0E", headerTextColor: "#FFFFFF",
 inputBackground: "#16161C", inputTextColor: "#FFFFFF",
 glassHeader: false, glassInput: false, accentGlow: true, hideBackgroundImage: false } });
```

Colors are given as a string “#RRGGBB” (or #AARRGGBB); values without “#” are ignored. The theme is also applied to the window (background, status bar, navigation bar). The first enabled plugin that returns a theme wins.

Network (NETWORK)

```
const r = await nano.net.request({ url, method: "POST", headers: {...},
 body: JSON.stringify(data), timeout: 30000, responseType: "json" /* text | json | binary */ });
// r = { status, headers, body, truncated }   binary -> body in base64
```

Only http/https, URL up to 4096 characters, methods GET/POST/PUT/DELETE/HEAD/PATCH, timeout 1–60 s (default 15), response limited to 10 MB (otherwise truncated: true), up to 5 redirects, each step is checked. The address is checked by IP after DNS: loopback, link-local, private networks (site-local), multicast and cloud metadata addresses (169.254.169.254) are blocked — requests to the local network are impossible. Errors arrive as { error: code }: permission_denied, invalid_args, invalid_url, domain_not_allowed, blocked_address, dns_resolution_failed, too_many_redirects, unsupported_method, request_failed.

Important: the network.domains/download.domains list is currently effectively treated as “any public domain” (in PluginCapabilities.parseDomains, when the block is present, “*” is returned). Do not rely on it as a restriction.

Files and downloads (FILES, DOWNLOAD)

```
await nano.files.save({ filename: "note.txt", data: "...", encoding: "utf8" /* or base64 */ });
const t = await nano.download.start({ url, filename: "model.bin" });
nano.task.status(t.id); nano.task.result(t.id); nano.task.cancel(t.id);
```

The file is saved to the plugin's isolated folder and copied to the shared “Files” section (NanoAI/Files). The name is sanitized (no paths, control characters or characters <>:"|?*\/, up to 180 characters); saving is limited to 25 MB, downloading to 200 MB.

Background tasks, actions, notifications

```
const t = await nano.task.start({ name: "sync" });
nano.task.progress(t.id, 40); nano.task.complete(t.id);   // or fail(id, msg)
nano.ui.action.register({ id: "run", title: "Run", handler: (ctx) => {...} });
nano.ui.notify({ title: "Done", message: "File saved" });
```

Embedding into the interface (UI_EXTEND)

```
nano.ui.extend({ slot: "message.actions", id: "explain", title: "Explain", icon: "bulb",
 prompt: "Explain more simply", handler: (ctx) => {...} });
```

Slot	Where it appears
message.actions	Under model messages next to “Copy”, “Repeat”
chat.empty	On the empty chat screen
settings.tab	A separate tab in settings (can open a panel)

Limits: up to 12 extensions per plugin, title up to 40 characters, id up to 64, prompt up to 400, HTML panel up to 200,000 characters. Allowed icon values: copy, share, edit/pencil, refresh/regenerate, like, dislike, trash/delete, search, bulb/idea, brain, bolt/flash, sparkles/magic, chat, heart, puzzle/plugin, play, download, settings, globe, rocket, palette, star/trophy.

Custom AI providers (AI_PROVIDER)

```
nano.ai.registerProvider({
 id: "my-ai", name: "My AI", models: [{id:"m1", name:"Model 1"}],
 settings: { baseUrl: { default: "https://api.example.com/v1", required: true }, apiKey: { required: true } },
 timeoutMs: 90000,
 generate: async (req) => {
 // req: { providerId, model:{id,name}, systemPrompt, messages:[{role,content}],
 //        params:{maxTokens,temperature,stream}, settings:{baseUrl,apiKey},
 //        request(opts), emit(text), isCancelled(), onCancel(fn) }
 const r = await req.request({ url: req.settings.baseUrl + "/chat", method: "POST", ... });
 if (r.status >= 400) throw nano.ai.fail("http", String(r.body), r.status);
 return text; } });
```

The provider **id** is [A-Za-z0-9_.-], 1–64 characters; up to 200 models; timeoutMs is limited to 5–300 s (default 120). The user enters the address and key in Settings → Plugins; the key is hidden in error messages (“***”). The API address must be a public https/http one. The return value is a string or an object with a text/content/reply field; partial output goes through req.emit(). Error kinds for nano.ai.fail: timeout, connection, unauthorized, rate_limited, not_found, content_filter, bad_reply, http. A complete working example: docs/example-openai-compatible.plugin.

4. What can and cannot be changed

4.1 In a plugin

Allowed

- Change the chat's appearance: colors, gradient, borders, bubble rounding, entrance animation, glass effect and glow (UI_THEME).
- Add quick actions, buttons under messages, blocks on the empty screen, tabs in settings (QUICK_ACTIONS, UI_EXTEND, UI_ACTION).
- Transform the user's text before sending and the response text before display (MESSAGE_HOOKS / MESSAGE_RENDER).
- Connect new models and providers, including non-standard APIs (AI_PROVIDER + NETWORK).
- Make requests to public APIs, download and save files, run background tasks, show notifications.
- Store the plugin's own settings (SETTINGS) and build HTML/CSS panels (CUSTOM_PANEL).
- Change allowed app settings: chat_bubble_style, haptics_enabled, font_scale (0.5–2), compact_mode, animations_enabled, legacy_gradient_bg, persona_text (up to 4,000 characters).

Not allowed (blocked by the app code)

- Read or change API keys, proxies, chats, history, export, the cloud account and any settings outside the APP_SETTINGS allowlist.
- Read other plugins' data: each plugin has its own storage and file folder.
- Access the local network, localhost, cloud metadata addresses; bypass this via redirect or DNS (the check is by IP at every step).
- Write files outside its own folder and the shared “Files” folder; use paths like ../, special characters in the name; exceed 25 MB (saving) and 200 MB (downloading).
- Execute JavaScript inside an HTML panel (JS is disabled in the panel) and read files/content providers from the WebView.
- Run native code, call Android APIs directly (there is no bridge other than nanoBridge); use anything that needs an undeclared permission: the call returns permission_denied or is silently ignored.
- Exceed limits: script and manifest 200 KB, network response 10 MB, 12 UI extensions, 200 models, network timeout 60 s.
- Change the app code, the Android permission list or system dialog colors — a plugin affects only what is described in section 3.4.

Not technically forbidden, but not advisable

- Using eval, new Function, obfuscation or large base64 blocks: the scanner will flag the plugin as suspicious and the user will get a warning.
- Requesting all permissions “just in case”: the user sees the permission list at installation, and extra permissions reduce trust.
- Sending chat history to your own server from AI_PROVIDER without explicitly warning the user: the provider receives the whole history while its model is selected, as well as its own key/address.
- Relying on the network.domains restriction — it currently does not work as an allowlist (see 3.5).

4.2 In the app itself (for developers)

Area	Change freely	Change carefully / do not change
Appearance	Colors, drawables, layouts, animations, font, icons, interface strings	Ids in layouts referenced by code (findViewById)
Translations	Values in assets/lang/*.tsv	Left column (Russian key): if you change Russian text in code, you must change the same key in all tsv files, otherwise the translation is lost
Providers/models	List of built-in models, BUILTIN_PROVIDERS list, error texts	Provider request formats without checking their API
Identifiers	applicationId, name, icon (required for a fork)	namespace com.youraicqa.app and NanoAI/... paths: tied to code, backup rules (backup_rules, data_extraction_rules, file_paths) and the .nanobackup format
Storage	New SharedPreferences keys	Names of existing keys and files — otherwise users' data is lost on update
Plugin system	New permissions and nano methods (add a has(...) check and a test)	Permission checks, PluginNetGuard (local address block), APP_SETTINGS allowlist, WebView sandbox — this is the security boundary
Data security	—	Key encryption (ServerKeyCipher), AES-256 export, the yb_* schema and RLS in Supabase
Native part	CMake parameters, llama.cpp version (check compatibility: llama_bridge.cpp has a compat layer)	JNI function signatures together with LlamaNative.kt

5. Forks and mods

A fork is your own repository based on this code. A mod is a modified build of the app. Both are permitted under the project license: the code is distributed under MIT (see the LICENSE file) — you may fork, modify and redistribute it, keeping the copyright notice and license text. The MIT license applies to the code; the font and third-party libraries have their own terms.

5.1 What you must change in a fork

What	Where	Why
applicationId and name	app/build.gradle.kts, res/values/strings	Otherwise the fork replaces the original on the device or conflicts with it
Signing key	GitHub Secrets: KEYSTORE_BASE64, KEYSTORE_PASSWORD, KEY_ALIAS, KEY_PASSWORD	Your own signature; without the secrets it builds with the debug key, which is unsuitable for publishing. Do not commit the key
Address transport	Transport.TRANSPORT_URL (and CONTROLLER_HOSTS/CLOUD_HOSTS)	The VonCure Studio service address on val.run is hardcoded. Until replaced, the fork contacts it once an hour. Set your own address or an empty string (then no polling is done)
Built-in models server	BuiltinModels.DEFAULT_REMOTE_URL	Also points to the VonCure Studio service; change it or keep only the bundled list
Cloud sessions	supabase/schema.sql + address/key via Transport (id “sessions”)	Create your own Supabase project and run schema.sql in the SQL Editor. If you do not need the cloud, make sure sync is disabled without an address (check Cloud.kt)
Links and “About” texts	AboutActivity, FaqContent, README	So you do not pass a fork off as the original or send users to someone else's support

5.2 Compatibility of forks and mods

- Backups (.nanobackup, .json) are portable between builds as long as the NanoAI/… folder names and the export format have not changed.
- .plugin plugins work in any fork with the same permission set and nano API version; if you added permissions or methods, document them — plugins using them will be incompatible with the original.
- If you changed the signature or applicationId, updating over the original is impossible: the user will need to export the account and import it into the fork.
- New languages: add a tsv to assets/lang and an entry in LanguageActivity.
- Sync with upstream on the branch, then re-run the unit tests (./gradlew test).

5.3 If you distribute a mod

- Give it its own name and icon, do not pass it off as an official build; keep the copyright notices and the license.
- Do not embed other people's API keys and proxies: the idea of the project is that keys belong to the user. Do not add hidden data sending; describe any network behavior that differs from the original in the README.
- Do not distribute builds with plugin protection disabled (permission checks, local address block): this protection is for users.
- Check the licenses of third-party parts: llama.cpp (MIT), AndroidX and PDFBox-Android (Apache-2.0), junrar (its own UnRAR license — forbids using the code to recreate the RAR compression algorithm), the font in assets/fonts (a Russified version of the VCR font by daymarius: the license for it is not confirmed; check the terms of the original and of the author of the Russification).
- Prefer a plugin over a fork if you only need a theme, buttons, a provider or a text hook: a plugin requires no rebuild and does not break updates.

5.4 Plugin or fork?

Task	Plugin	Fork
Custom theme and colors	yes (UI_THEME)	—
New model / provider API	yes (AI_PROVIDER)	if a native/streaming protocol is needed
“Explain” button, request templates	yes (UI_EXTEND, QUICK_ACTIONS)	—
Post-processing of responses	yes (MESSAGE_RENDER)	—
New screen/mode with its own logic	limited (HTML panel without JS)	yes

Task	Plugin	Fork
Access to chats, keys, files outside the plugin folder	not possible	yes, but the responsibility is yours
Different native engine / llama.cpp version	not possible	yes

6. Security and privacy — in brief

- Provider keys are stored on the device and sent only to their own provider; they are not included in chat export; in account export they are included in encrypted form (AES-256, user password; a forgotten password cannot be recovered).
- Keys of custom servers are encrypted and bound to the server id; the server certificate can be pinned, and the app notices a certificate change.
- The cloud stores only the account identifier (the secret — as a SHA-256), the session list and device labels; access to the tables is only through yb_* functions (RLS enabled, no policies).
- Transport accepts addresses only for allowed domains, so a forged response cannot send secrets to a foreign server.
- Agent and web view: blocking of local addresses and dangerous schemes, no access to geolocation/camera/microphone, confirmation of dangerous actions.
- The NanoLog log does not contain keys or message texts.
- If you find a vulnerability in the plugin sandbox, do not publish an exploit: report it privately to the project author.

7. Build and release

Locally

```
git clone --depth 1 https://github.com/ggml-org/llama.cpp app/src/main/cpp/llama.cpp
./gradlew assembleDebug   # APK: app/build/outputs/apk/debug/
./gradlew test            # unit tests
```

Requires JDK 17, Android SDK 34+, NDK 27.0.12077973, CMake 3.22.1.

GitHub Actions (build-apk.yml)

- Runs on push to main/master and manually (workflow_dispatch).
- Finds a *.zip (YourbitAIv3.zip) in the repository root, unpacks it, determines the project root by build.gradle(.kts) and copies it to the root.
- Restores the signing key from the KEYSTORE_BASE64 secret, clones llama.cpp, installs JDK 17, the Android SDK, NDK and CMake.
- Builds a debug APK (signed with your key if the secrets are set), copies it to out/YourbitAI-debug.apk.
- Tries to upload the APK to gofile.io, then catbox.moe, and writes the link to the Summary; if the hosts are unavailable, publishes a pre-release “build-N” in Releases; additionally saves an artifact for 1 day.
- To publish a release in Releases manually: gh release create vX.Y.Z out/YourbitAI-debug.apk with a change description.